

面向 LLM / Agent Systems 科研的深度学习零基础课

第 05 讲

Linear / Weight / Bias 核心概念速查卡

以后读 PyTorch、Transformer 与论文 Methods 时最常用的 Shape、参数数量与记法对照。

核心概念速查卡

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

1. 一句话模型

核心概念

Linear Layer 的核心是：用 **Weight** 对输入做线性组合，再加 **Bias** 做平移。

NumPy 行向量约定：

$$Y = XW + b$$

2. Shape 模板

若：

$$X : (B, D_{in})$$

映射到 D_{out} ，则：

$$W : (D_{in}, D_{out}), \quad b : (D_{out},)$$

$$Y : (B, D_{out})$$

例：

$$(32, 768) @ (768, 3072) + (3072,) \rightarrow (32, 3072)$$

3. 一个输出怎么来

第 j 个输出：

$$y_j = x \cdot W_{:,j} + b_j$$

即：输入向量与 Weight 第 j 列做 Dot Product，再加第 j 个 Bias。

4. 参数数量

$$\#W = D_{in}D_{out}$$

$$\#b = D_{out}$$

$$\#parameter = D_{in}D_{out} + D_{out}$$

5. Weight 与 Bias 的作用直觉

对象	核心作用
Weight	决定输入各维怎样被混合、放大、减弱并投影成输出
Bias	对每个输出维度额外加一个可学习平移量
Parameter	训练过程中会被 Gradient/Optimizer 更新的数字

6. 严格术语

$$y = Wx$$

严格叫 Linear Transformation。

$$y = Wx + b, \quad b \neq 0$$

严格叫 Affine Transformation，但深度学习框架仍常命名为 Linear Layer。

7. 三种常见记法不要混

场景	Weight Shape	前向形式
列向量数学教材	(D_{out}, D_{in})	$y = Wx + b$
NumPy 行向量课程约定	(D_{in}, D_{out})	$y = xW + b$
PyTorch nn.Linear 存储	(D_{out}, D_{in})	$y = xW^T + b$

常见误区 / 风险

永远优先看 Shape。不要因为公式里 W 在左或在右，就判断别人“写错了”。

8. Transformer 连接

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

本质：同一个输入 X 经过三套不同 Weight 的 Linear Projection。

FFN 常见：

$$D \rightarrow D_{ff} \rightarrow D$$

也是 Linear + Activation + Linear。

9. PyTorch 速查

```
import torch.nn as nn

layer = nn.Linear(768, 3072)
print(layer.weight.shape) # (3072, 768)
print(layer.bias.shape)   # (3072,)
```

本节检查点

给出 $D_{in} = 128, D_{out} = 256$ ：采用 $X@W + b$ 约定时，必须能立即写出 $W : (128, 256)$ 、 $b : (256,)$ 。